

```
import os
import json
from collections import defaultdict
from typing import Dict, List, Optional
```

```
from reportlab.platypus import SimpleDocTemplate, Paragraph, Spacer
from reportlab.lib.styles import getSampleStyleSheet
```

```
# -----
# 1. CLASS MAP (정석 버전)
# -----
CLASS_MAP: Dict[str, int] = {
    "Car": 1,
    "Van": 1,
    "Truck": 1,
    "Pedestrian": 2,
    "Cyclist": 3,
}
```

```
IGNORE_CLASSES = {"DontCare", "Misc", "Tram", "Person_sitting"}
```

```
# -----
# 2. PARSER
# -----
def parse_line(line: str) -> Optional[List]:
    parts = line.strip().split()

    if len(parts) < 8:
        return None

    cls = parts[0]
```

```

if cls in IGNORE_CLASSES:
    return None

if cls not in CLASS_MAP:
    return None

try:
    x1 = float(parts[4])
    y1 = float(parts[5])
    x2 = float(parts[6])
    y2 = float(parts[7])
except:
    return None

return [cls, CLASS_MAP[cls], x1, y1, x2, y2]

```

```

# -----
# 3. CONVERSION + STATS
# -----
def convert_and_analyze(input_dir, output_dir):

    os.makedirs(output_dir, exist_ok=True)

    stats = {
        "total_objects": 0,
        "kept_objects": 0,
        "class_count": defaultdict(int)
    }

    files = [f for f in os.listdir(input_dir) if f.endswith(".txt")]

    for f in files:
        in_path = os.path.join(input_dir, f)

```

```

out_path = os.path.join(output_dir, f)

out_lines = []

with open(in_path, "r") as fin:
    for line in fin:
        stats["total_objects"] += 1

        parsed = parse_line(line)

        if parsed is None:
            continue

        cls, cid, x1, y1, x2, y2 = parsed

        stats["kept_objects"] += 1
        stats["class_count"][cls] += 1

        out_lines.append(f"{cid} {x1} {y1} {x2} {y2}")

with open(out_path, "w") as fout:
    fout.write("\n".join(out_lines) + ("\n" if out_lines else ""))

return stats

```

```
# -----
```

```
# 4. REPORT GENERATOR (PDF)
```

```
# -----
```

```
def generate_report(stats, save_path="report.pdf"):
```

```
    doc = SimpleDocTemplate(save_path)
```

```
    styles = getSampleStyleSheet()
```

```

content = []

content.append(Paragraph("KITTI Dataset Preprocessing Report",
styles["Title"]))
content.append(Spacer(1, 12))

content.append(Paragraph(f"Total objects: {stats['total_objects']}",
styles["Normal"]))
content.append(Paragraph(f"Kept objects: {stats['kept_objects']}",
styles["Normal"]))
content.append(Spacer(1, 12))

content.append(Paragraph("Class Distribution:", styles["Heading2"]))

for k, v in stats["class_count"].items():
    content.append(Paragraph(f"{k}: {v}", styles["Normal"]))

content.append(Spacer(1, 12))

discard_rate = 1 - (stats["kept_objects"] / max(stats["total_objects"],
1))
content.append(
    Paragraph(f"Discard rate: {discard_rate:.2%}", styles["Normal"])
)

doc.build(content)

# -----
# 5. MAIN
# -----
def main():

    input_dir = "labels_raw"

```

```
output_dir = "labels_clean"
```

```
print("[1] Converting KITTI labels...")
```

```
stats = convert_and_analyze(input_dir, output_dir)
```

```
print("[2] Saving stats...")
```

```
with open("stats.json", "w") as f:
```

```
    json.dump(stats, f, indent=2)
```

```
print("[3] Generating PDF report...")
```

```
generate_report(stats)
```

```
print("DONE → report.pdf 생성 완료")
```

```
if __name__ == "__main__":
```

```
    main()
```